Academia de Studii Economice din București
Facultatea de Cibernetică, Statistică și Informatică Economică

PROIECT

Aplicație care permite luarea unei decizii la nivel colectiv

Coordonator:
Cadru didactic asociat CRACEA ALEXANDRU-STEFAN

Nume și prenume student:
CRISTEA ELENA-ALEXIA
Seria: D-INF
Grupa: 1017

Cuprins

1. CERINȚA PROIECTULUI

2. PROIECTAREA ȘI DEZVOLTAREA APLICAȚIEI

2.1. Inițializarea aplicației și crearea fișierelor esențiale

2.2. Funcții auxiliare

2.2.1. Introducere variabile fișiere + inițializare automată

2.2.2. Funcția `*valideaza_cod()*`

2.2.3. Funcția `*afiseaza_optiuni()*`

2.3. Funcții principale

2.3.1. Funcția `*voteaza()*`

2.3.2. Funcția `*afiseaza_rezultate*`

2.3.3. Meniul principal al aplicației

2.4. Validări și restricții

2.4.1. Validări generale

2.4.2. Restricții specifice funcțiilor

3. DEMONSTRAȚIE DE FUNCȚIONARE

3.1. Votare cu cod valid și opțiune validă

3.2. Vizualizare rezultate

3.3. Tentativă de vot dublu (cu același cod)

3.4. Introducerea unui cod invalid

4. FUNCȚIONALITĂȚI SUPLIMENTARE + CE AȘ FI PUTUT ÎMBUNĂTĂȚI

1. CERINȚA PROIECTULUI

Tema 2: Aplicație care permite luarea unei decizii la nivel colectiv.

Scopul proiectului este realizarea unei aplicații în Bash care simulează un sistem de luare a deciziilor la nivel colectiv, prin vot anonim, în care fiecare utilizator are un cod unic de votare.

Funcționalități:

- Importă codurile valide dintr-un fișier (coduri.txt);
- Permite introducerea codului de vot și verifică dacă este valid + nefolosit;
- Prezintă o listă de opțiuni (ex: A, B, C);
- Înregistrează votul într-un fișier (rezultate.txt);
- Marchează codul ca folosit (ex: mută codul în coduri_folosite.txt);
- Oferă opțiunea de a vizualiza rezultatele (număr de voturi per opțiune);

2. PROIECTAREA ȘI DEZVOLTAREA APLICAȚIEI

Proiectul a fost dezvoltat și testat pe un sistem macOS, utilizând aplicația Terminal pentru scrierea și rularea scriptului Bash. Toate comenzile, validările și funcționalitățile au fost implementate direct în această interfață de linie de comandă.

2.1. Inițializarea aplicației și crearea fișierelor esențiale

Descriere:

Această etapă constă în pregătirea fișierelor necesare pentru funcționarea aplicației. Se utilizează comanda touch pentru a crea cele trei fișiere esențiale:

- coduri.txt – conține codurile valide;
- coduri_folosite.txt – conține codurile care au fost deja utilizate la vot;
- rezultate.txt – conține rezultatele voturilor exprimate.

De asemenea, este creat și scriptul principal (votare.sh) care va conține logica aplicației.

Funcționalități:

- Asigură crearea fișierelor necesare;
- Evită erori legate de fișiere lipsă;
- Creează infrastructura de bază pentru sistemul de vot;



~ Crearea fișierelor necesare aplicației, inclusiv scriptul principal votare.sh, folosind comenzile touch și ls în terminal. ~

Concluzie: Această etapă de inițializare este esențială pentru buna funcționare a aplicației, deoarece elimină riscul de eroare la rulare și oferă utilizatorului un punct de plecare sigur și organizat.

2.2. Funcții auxiliare

2.2.1 Introducere variabile fișiere + inițializare automată

Descriere:

În prima parte a scriptului din votare.sh, au fost definite variabile pentru a face codul mai organizat și ușor de întreținut. Aceste variabile rețin numele fișierelor esențiale folosite în sistemul de vot. Acest lucru permite folosirea rapidă și flexibilă a fișierelor în tot scriptul, fără a repeta numele lor text direct în comenzi.

```
#!/bin/bash

# ====================================================
# CERINTA 1: Importa codurile valide din coduri.txt
# ====================================================

CODURI_FILE="coduri.txt"
FOLOSITE_FILE="coduri_folosite.txt"
REZULTATE_FILE="rezultate.txt"
OPTIUNI=("A" "B" "C")
```

De asemenea, imediat după definirea acestor variabile, se face verificarea și inițializarea automată a fișierelor. Prin aceste comenzi, scriptul se asigură că fișierele necesare există înainte de orice acțiune (vot, verificare, salvare), prevenind astfel erori la rulare.

```
touch "$CODURI_FILE"
touch "$FOLOSITE_FILE"
touch "$REZULTATE_FILE"
```

Funcționalități:

- Centralizează toate fișierele importante în variabile reutilizabile;
- Permite modificarea rapidă a fișierelor fără a edita codul în mai multe locuri;
- Creează fișierele necesare automat, dacă lipsesc;
- Oferă un punct de plecare stabil aplicației la fiecare execuție;

2.2.2. Funcția `*valideaza_cod()*`

Descriere:

Funcția are rolul de a valida codul introdus de utilizator, verificând dacă acesta este valid (există in fișierul coduri.txt) și dacă nu a mai fost folosit (nu se află în coduri_folosite.txt). Dacă oricare dintre aceste condiții nu este îndeplinită, codul este considerat invalid sau deja utilizat.

Funcționalități:

- Previne votarea multiplă;
- Asigură că doar utilizatorii autorizați pot vota;
- Afișează mesaje informative pentru utilizator în cazul erorilor;

```
# ========================================================
# CERINTA 2: Permite introducerea codului de vot(cheile
#            de acces) si verifica daca este valid
#            si nefolosit
# ========================================================

function valideaza_cod {
    cod="$1"
    if grep -Fxq "$cod" "$CODURI_FILE"; then
        if grep -Fxq "$cod" "$FOLOSITE_FILE"; then
            echo "Codul a fost deja folosit."
            return 1
        else
            return 0
        fi
    else
        echo "Oops! Codul este invalid."
        return 1
    fi
}
```

2.2.3. Funcția `*afiseaza_optiuni()*`

Descriere:
Această funcție afișează în terminal opțiunile disponibile pentru vot, definite în array-ul OPTIUNI. Fiecare opțiune este afișată pe câte un rând nou, într-un format clar și ușor de înțeles. Funcția poate fi adaptată ușor pentru a afișa orice listă de opțiuni, nu doar A, B, C.

Funcționalități:
- Oferă utilizatorului vizibilitate asupra opțiunilor disponibile;
- Permite extinderea ușoară a opțiunilor;
- Contribuie la o interfață interactivă și prietenoasă;

```
# ========================================================
# CERINTA 3: Prezinta o lista de optiuni (ex: A, B, C)
# ========================================================

function afiseaza_optiuni {
    echo "Optiuni disponibile pentru vot:"
    for opt in "${OPTIUNI[@]}"; do
        echo " - $opt"
    done
}
```

2.3. Funcții principale

2.3.1. Funcția `*voteaza()*`

Descriere:

Funcția gestionează tot procesul de vot:

- cere introducerea codului;
- Verifică dacă codul este valid și nu conține caractere speciale;
- afișează opțiunile de vot disponibile;
- Permite utilizatorului să introducă opțiunea dorită de vot;
- verifică dacă votul este valid;
- înregistrează votul în rezultate.txt și codul în coduri_folosite.txt.

Funcționalități:

- Asigură protecție împotriva codurilor invalide sau goale;
- Transformă inputul în format standard (majuscule, fără spații inutile);
- Oferă feedback imediat asupra succesului sau eșecului votului;
- Gestionează salvarea datelor într-un mod persistent;
- Se asigură că fiecare vot este salvat și asociat cu un cod valid, care nu poate fi reutilizat

```
# ==================================================
# CERINTELE 4, 5: Inregistreaza votul in rezultate.txt
#                 si marchează codul in coduri_folosite.txt
# ==================================================

function voteaza {
    read -p "Introdu codul tau de votare: " cod
    cod=$(echo "$cod" | xargs)
    cod=$(echo "$cod" | tr '[:lower:]' '[:upper:]')

    if [[ -z "$cod" ]]; then
        echo "Nu ai introdus niciun cod. Incearca din nou."
        return
    fi

    if ! [[ "$cod" =~ ^[A-Za-z0-9]+$ ]]; then
        echo "Codul contine caractere invalide. Incearca din nou."
        return
    fi
    valideaza_cod "$cod" || return

    afiseaza_optiuni
    read -p "Introdu optiunea ta (A, B sau C): " vot
    if [[ -z "$vot" ]]; then
        echo "Nu ai introdus nicio optiune. Votul nu a fost inregistrat. Incearca din nou."
        return
    fi

    vot_upper=$(echo "$vot" | xargs | tr '[:lower:]' '[:upper:]')

    if [[ " ${OPTIUNI[*]} " == *" $vot_upper "* ]]; then
        # CERINTA 4: Salvare vot in rezultate.txt
        echo "$vot_upper" >> "$REZULTATE_FILE"

        # CERINTA 5: Salvare cod in coduri_folosite.txt
        echo "$cod" >> "$FOLOSITE_FILE"

        echo "Votul a fost inregistrat cu succes."
    else
        echo "Oops! Optiune invalida. Incearca din nou"
    fi
}
```

2.3.2. Funcția *`afiseaza_rezultate()`*

Descriere:

Această funcție parcurge opțiunile de vot și afișează câte voturi a primit fiecare. În plus, calculează totalul voturilor exprimate folosind comanda wc -l.

Funcționalități:

- Oferă transparență asupra rezultatelor votului;
- Permite utilizatorilor să vadă în timp real câte voturi are fiecare opțiune;
- Calculează numărul total de voturi;
- Asigură afișarea corectă a fiecărei opțiuni și a voturilor primite;

```
# ======================================================
# CERINTA 6: Ofera optiunea de a vizualiza rezultatele
# ======================================================

function afiseaza_rezultate {
    echo "Rezultate vot:"
    for opt in "${OPTIUNI[@]}"; do
        count=$(grep -c "^$opt$" "$REZULTATE_FILE")
        echo "$opt: $count voturi"
    done

    total=$(wc -l < "$REZULTATE_FILE")
    echo "Totalul voturilor exprimate este: $total"
}
```

Concluzie: Această funcție contribuie la transparența procesului de vot și permite verificarea rezultatului în orice moment.

2.3.3. Meniul principal al aplicației

Descriere:

Structura principală a aplicației este un meniu afișat în buclă infinită (while true) care permite utilizatorului să aleagă între:

1. a vota;
2. a vizualiza rezultatele;
3. a ieși din aplicație.

Funcționalități:

- Asigură o interfață clară, simplă și ușor de folosit;
- Permite utilizatorului să interacționeze cu aplicația fără să știe programare;
- Menține aplicația activă până la ieșirea explicită (buclă while true);
- Gestionează eficient acțiunile posibile prin comenzi numerotate;
- Oferă feedback clar pentru opțiuni greșite, prevenind blocarea aplicației;

```
# ========================================================
# Meniu principal – interfata aplicatiei
# ========================================================

while true; do
    echo ""
    echo "=== Sistem de Vot Colectiv ==="
    echo "1. Voteaza"
    echo "2. Vizualizeaza rezultatele voturilor"
    echo "3. Iesire din aplicatie"
    read -p "Alege o optiune: " alegere

    case "$alegere" in
        1) voteaza ;;
        2) afiseaza_rezultate ;;
        3) echo "Ai iesit din aplicatie. La revedere!"; exit 0 ;;
        *) echo "Oops! Optiune invalida. Incearca din nou!" ;;
    esac
done
```

2.4. Validări și restricții

2.4.1. Validări generale

Descriere:

Aplicația include validări de bază pentru inputul introdus de utilizator, cu scopul de a preveni erori și comportamente neașteptate. Aceste validări sunt aplicate indiferent de funcția folosită și includ:

- Verificarea dacă codul este gol: -z "$cod"
- Verificarea dacă codul conține doar caractere valide (litere și cifre): ^[A-Za-z0-9]+ $
- Verificarea dacă votul este gol
- Verificarea dacă opțiunea aleasă nu este A, B sau C

Funcționalități:

- Protejează aplicația împotriva introducerii de date incorecte sau deformate;
- Asigură că logica de votare rămâne corectă;
- Evită blocaje, erori sau comportamente neprevăzute ale scriptului;

2.4.2. Restricții specifice funcțiilor

Descriere:

Fiecare funcție din aplicație are propriile condiții de validare, astfel încât utilizatorul să nu poată folosi aplicația într-un mod incorect sau nepermis. Aceste restricții sunt:

- *valideaza_cod* – nu permite utilizarea codurilor inexistente sau care au fost deja folosite;
- *voteaza* – nu acceptă opțiuni invalide (doar A, B sau C sunt permise);
- Nu se poate vota de două ori cu același cod;
- Dacă se introduce un cod gol sau cu caractere speciale, aplicația afișează un mesaj de eroare;
- Meniul principal nu acceptă alte comenzi decât 1, 2 sau 3 — orice alt input este respins cu un mesaj „Opțiune invalidă”.

Funcționalități:

- Asigură controlul complet asupra fluxului de utilizare;
- Ajută la păstrarea corectitudinii și a funcționării stabile a aplicației;
- Previne abuzurile (vot dublu, opțiuni false etc.);
- Oferă mesaje clare în caz de eroare;

3. DEMONSTRAȚIE DE FUNCȚIONARE

3.1. Votare cu cod valid și opțiune validă

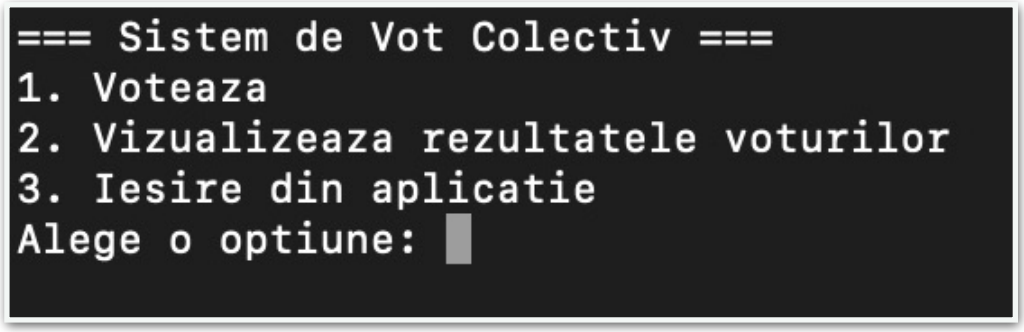
*Scenariu***:** Utilizatorul pornește aplicația, introduce un cod valid (spre exemplu COD1000) și o opțiune validă de vot (spre exemplu A). Concluzie: Votul este înregistrat cu succes.

Pas 1: Adăugarea codurilor în coduri.txt. Deschidem fișierul cu comanda pico și apoi adăugăm codurile.



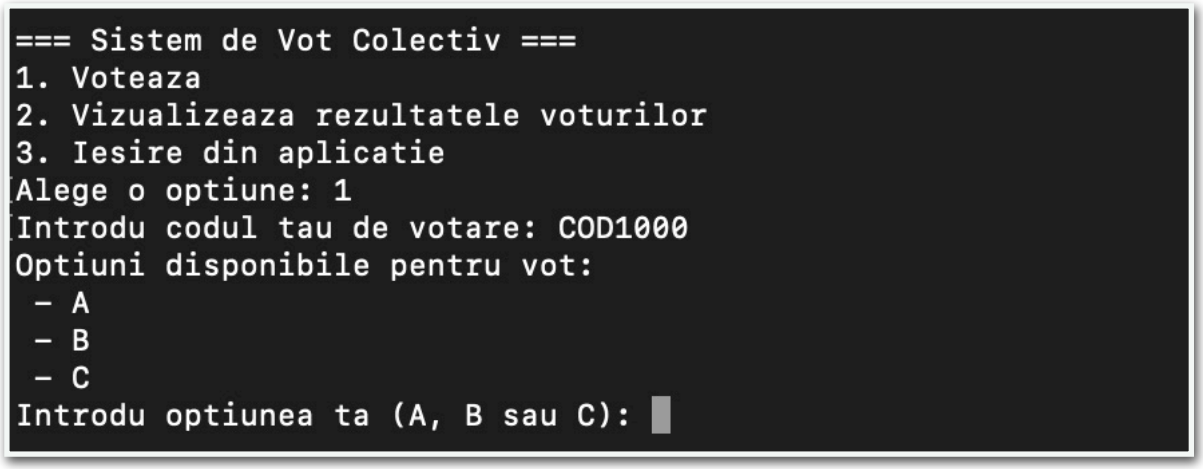
Pas 2: Rulăm aplicația

Comandă rulată în terminal: ./votare.sh

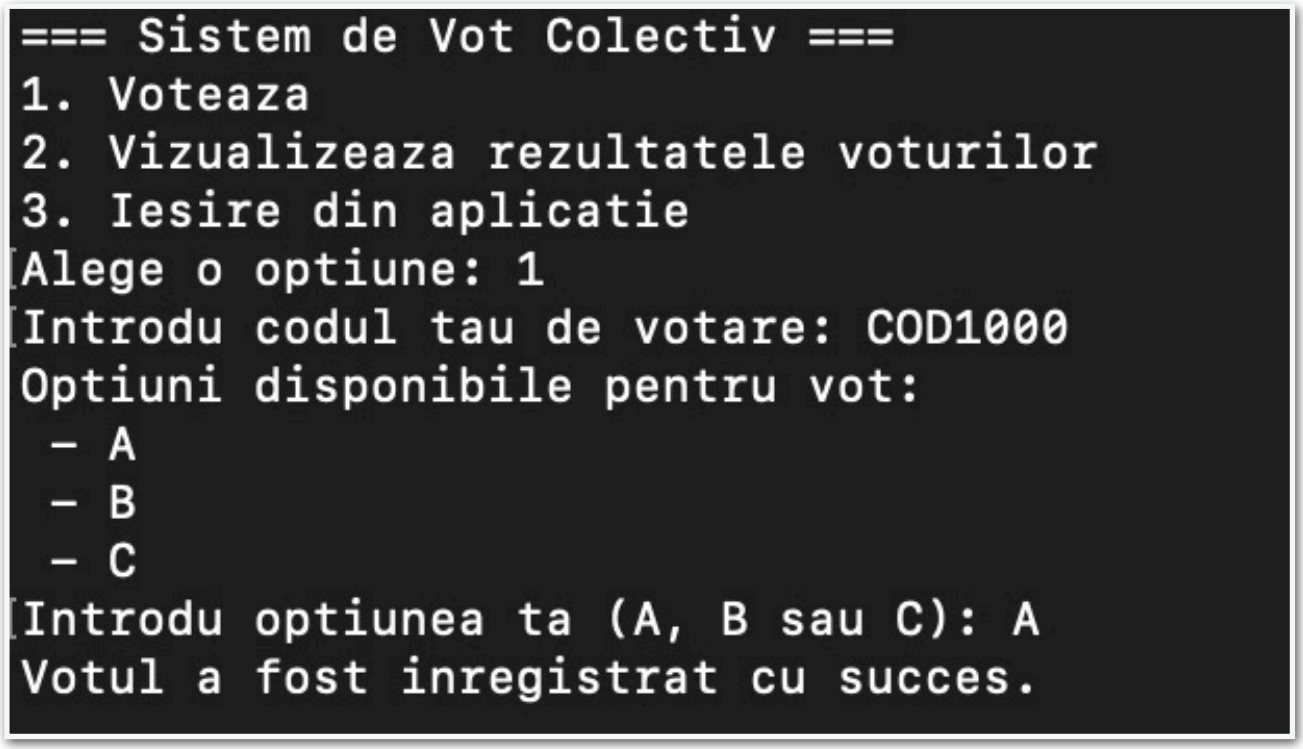


Pas 3: Alegem opțiunea 1 (Votează)

Pas 4: Introducem un cod valid (spre exemplu COD1000), apoi vor apărea opțiunile disponibile pentru vot (A, B, C).



Pas 5: Introducem opțiunea de vot (spre exemplu A), iar aplicația confirmă votul.



~ Votul a fost exprimat cu succes folosind codul COD1000 ~

Concluzie: În acest exemplu, am demonstrat cum un cod valid, nefolosit, permite exprimarea unui vot, iar aplicația răspunde corect cu un mesaj de confirmare.

Pentru testarea completă a funcționalității aplicației, au fost utilizate toate codurile existente în fișierul coduri.txt, fiecare exprimând câte un vot valid, asociat unei opțiuni dintre A, B sau C.

Tabelul de mai jos confirmă că fiecare cod a fost folosit o singură dată și fiecare vot a fost înregistrat corect:

COD INTRODUS	OPȚIUNE ALEASĂ
COD111	A
COD222	B
COD333	C
COD444	B
COD555	A
COD666	A
COD777	A
COD888	B
COD999	C
COD1000	A

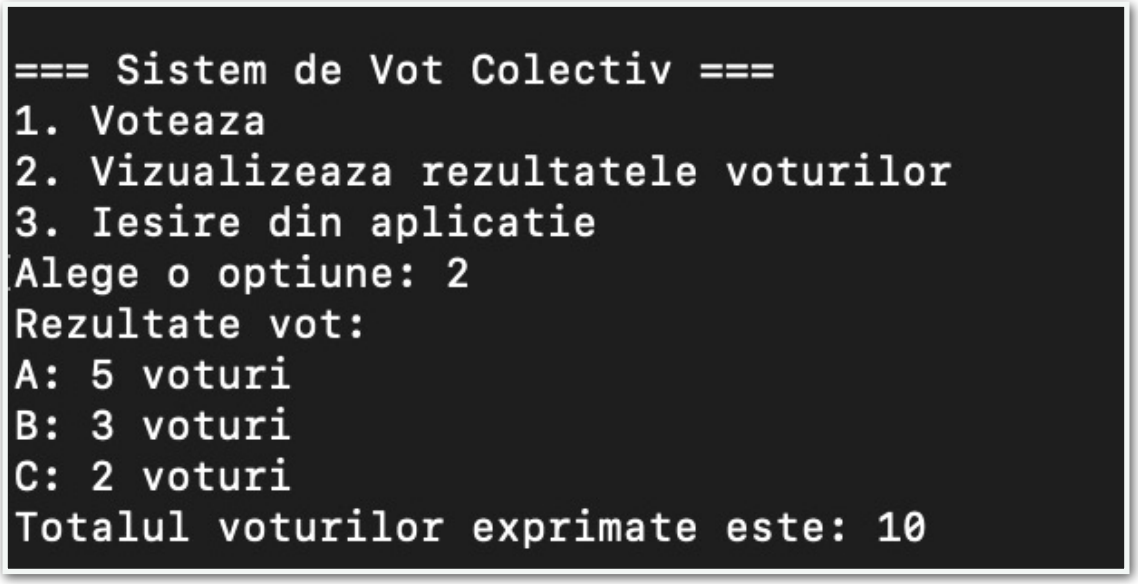
Mai jos este structura fișierului rezultate.txt, după înregistrarea tuturor voturilor. Fiecare linie corespunde unui vot exprimat, într-un format simplu (A, B, C), exact în ordinea în care au fost introduse codurile.



3.2. Vizualizare rezultate

Scenariu: Utilizatorul selectează opțiunea 2 din meniu pentru a vizualiza rezultatele voturilor exprimate până în acel moment.

După exprimarea tuturor voturilor, fișierul rezultate.txt conține exact zece înregistrări, câte una pentru fiecare cod, în ordinea exprimării.



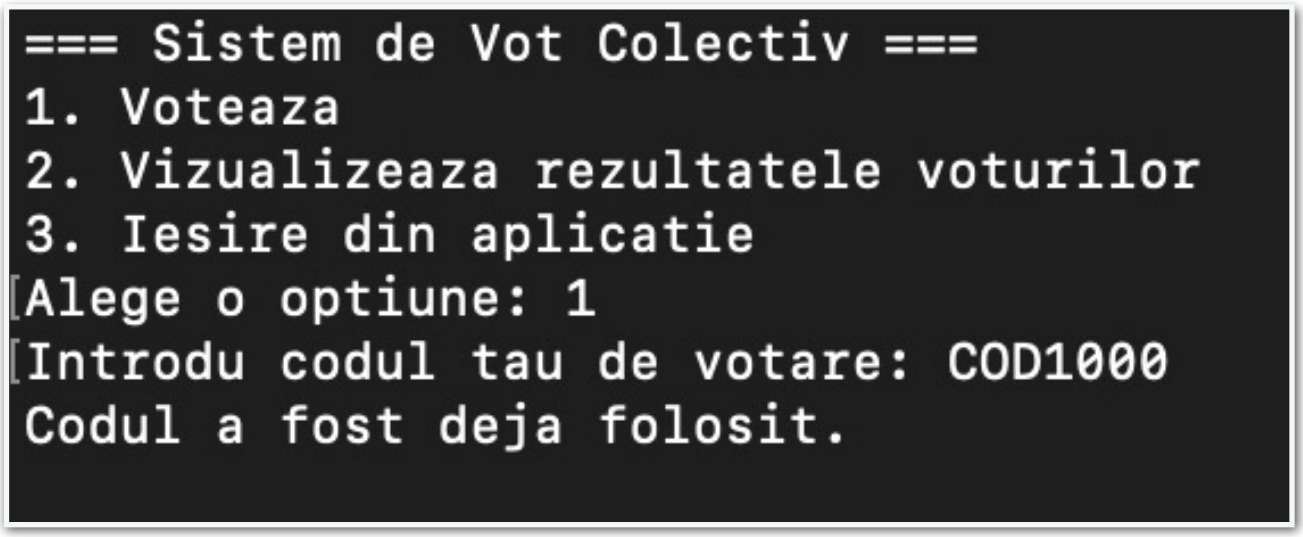
~ Afișarea rezultatului final al voturilor exprimate ~

Concluzie: Afișarea rezultatului final corespunde cu voturile exprimate anterior.

3.3 TENTATIVĂ DE VOT DUBLU

După exprimarea unui vot valid cu codul COD1000, s-a realizat un nou test pentru a verifica dacă aplicația permite reutilizarea aceluiași cod.

Aplicația a detectat că același cod a mai fost folosit anterior, motiv pentru care a refuzat exprimarea unui nou vot. Astfel, se asigură că fiecare utilizator poate vota o singură dată, conform regulilor unui sistem corect de vot.



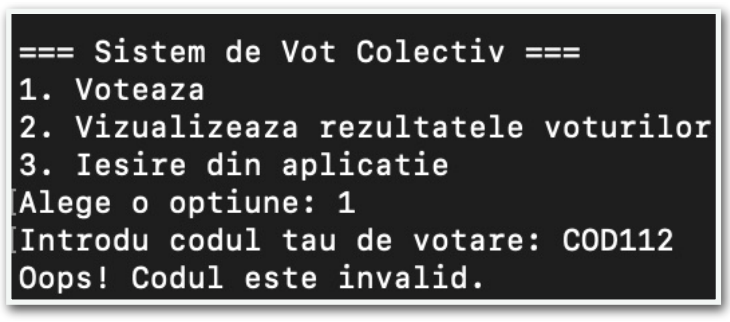
3.4. Introducerea unui cod invalid

Descriere:

În acest test, utilizatorul încearcă să voteze folosind un cod care nu există în fișierul coduri.txt (ex: COD112). Aplicația verifică validitatea codului și, dacă nu îl găsește în lista de coduri autorizate, afișează un mesaj de eroare care informează utilizatorul că acel cod este invalid.

Funcționalități:

- Verifică dacă codul introdus se află în lista codurilor valide;
- Afișează un mesaj clar în cazul unui cod greșit sau inexistent;
- Împiedică înregistrarea unui vot neautorizat;
- Contribuie la securitatea și integritatea procesului de vot.



4. FUNCȚIONALITĂȚI SUPLIMENTARE + CE AȘ FI PUTUT ÎMBUNĂTĂȚI

Chiar dacă aplicația funcționează conform cerințelor, m-am gândit că aș mai fi putut adăuga câteva lucruri utile, care ar fi făcut-o mai clară și mai ușor de folosit:

1. **Rezultatele pe cod, nu doar pe opțiune**

În loc să apară doar A, B sau C în fișierul rezultate.txt, ar fi fost mai clar dacă apărea și codul care a votat, de exemplu: COD111: A.

2. **Data și ora la fiecare vot**

Ar fi fost interesant ca fiecare vot salvat să aibă și data și ora când a fost făcut. Asta ajută dacă vrei să vezi când a votat fiecare.

3. **Filtru pentru afișare pe opțiuni**

Mi s-a părut că ar fi util să poți vedea doar voturile pentru A, sau doar pentru B, nu toate deodată, mai ales dacă sunt multe.